

TP 15 — Dichotomie : l'implémenter et la mesurer

Semaine 15 ■ Fichiers dans NSI/TP15/ ■ 2 h

Objectifs

- Implémenter la dichotomie **avec la méthode de la semaine 13** (docstring, tests, corps)
- Compter les comparaisons et vérifier le $\log_2 n$ expérimentalement
- Mettre côte à côte séquentielle et dichotomie sur un million d'éléments

Partie A — Implémentation testée — `dicho.py`

(45 min)

1. Écrire le jeu de tests **d'abord** :

```
def test_dichotomie():
    t = [2, 5, 7, 11, 13, 17, 23]
    assert dichotomie(t, 11) == 3
    assert dichotomie(t, 2) == 0           # premier
    assert dichotomie(t, 23) == 6         # dernier
    assert dichotomie(t, 4) is None       # absent (milieu)
    assert dichotomie(t, 1) is None       # absent (avant)
    assert dichotomie(t, 99) is None      # absent (après)
    assert dichotomie([], 5) is None      # vide
    assert dichotomie([7], 7) == 0        # un seul
    print("test_dichotomie : OK")
```

2. Implémenter `dichotomie(t, x)` (cours) et passer au vert.
3. La rejouer à la **main** : sur papier, dérouler `dichotomie(t, 13)` en notant (gauche, droite, milieu).
4. ★ Casser volontairement : remplacer `gauche = milieu + 1` par `gauche = milieu`, relancer les tests. Lequel part en boucle infinie ? (Couper avec Ctrl-C, noter, remettre.)

Partie B — Compter les comparaisons — `compte.py`

(35 min)

1. Copier `dichotomie` en `dichotomie_comptee(t, x)` qui renvoie (indice, nb_comparaisons) (compter une comparaison par tour de boucle).
2. Pour n dans [100, 1000, 10000, 100000, 1000000] : construire `t = list(range(n))` et afficher le nombre de comparaisons pour chercher -1 (absent = pire cas).
3. Comparer avec $\log_2 n$ (en Python : `n.bit_length()`). Conclusion ?
4. ★ Vérifier la propriété « taille doublée = une comparaison de plus » entre n et $2n$.

Partie C — Le duel — `duel.py`

(30 min)

Reprendre le chronométrage du TP 11 (`time.perf_counter`) :

1. `t = list(range(1_000_000))` ; chronométrer la recherche de -1 (pire cas) par `contient` (séquentielle, TP 11) puis par `dichotomie`.
2. Chronométrer **1000 recherches** de valeurs aléatoires avec chacune. Facteur d'accélération ?
3. Rédiger en commentaire la morale du cours : dans quel cas vaut-il la peine de trier d'abord ?

Partie D — Défi ★★★

(10 min)

`racine_entiere(n)` — le plus grand entier r tel que $r^2 \leq n$, par dichotomie sur l'intervalle $[0, n]$ (sans tableau!). Tests : `racine_entiere(17)` $\rightarrow$ 4, `racine_entiere(16)` $\rightarrow$ 4, `racine_entiere(0)` $\rightarrow$ 0.